

IFVG Sniper Model — Pine Script v6

TradingView indicator · defaults tuned for the 15-minute chart · 583 lines. Paste into Pine Editor → Save → Add to chart.

```
1  //@version=6
2  // _____
3  // IFVG Sniper Model (defaults tuned for the 15-minute chart)
4  // 1. Liquidity sweep — price wicks through a swing / previous-day high-low and closes back inside
5  // 2. V-shape recovery — fast displacement away from the sweep extreme (× ATR)
6  // 3. CISD — close through the open of the delivery series that made the extreme
7  // 4. IFVG — an opposing FVG is closed through by a displacement candle → sniper zone
8  // 5. Retest + sweep — price returns into the IFVG, sweeps minor liquidity and holds → entry
9  // 6. Targets — fixed 1R / 2R (default) with a draw on liquidity required beyond TP1
10 // Filters — higher-timeframe bias, London / New York killzones, max stop size
11 // _____
12 indicator("IFVG Sniper Model", "IFVG Sniper", overlay = true, max_bars_back = 500,
13     max_lines_count = 500, max_labels_count = 500, max_boxes_count = 500)
14
15 // _____ Inputs _____
16 const string G0 = "0 · High-Probability Filters"
17 useHtf = input.bool(true, "Trade with higher-timeframe bias", group = G0,
18     tooltip = "Longs only when the HTF close is above a rising EMA, shorts only below a falling EMA (last closed HTF bar, no repaint).")
19 htfTf = input.timeframe("60", "HTF", group = G0, inline = "htf")
20 htfEmaLen = input.int(50, "EMA", minval = 1, group = G0, inline = "htf")
21 useKz = input.bool(true, "Entries only inside killzones", group = G0)
22 kz1 = input.session("0200-0500", "London", group = G0, inline = "kz")
23 kz2 = input.session("0700-1100", "New York", group = G0, inline = "kz")
24 kzTz = input.string("America/New_York", "Killzone timezone", group = G0)
25 dispBody = input.float(0.5, "Min body % of IFVG inversion candle", minval = 0, maxval = 1, step = 0.05,
26     group = G0,
27     tooltip = "The candle that closes through the FVG must be a displacement candle (body / range). 0 disables.")
28 maxSlAtr = input.float(2.0, "Skip entry if stop > N × ATR", minval = 0.2, step = 0.1, group = G0)
29 minSlAtr = input.float(0.3, "Minimum stop size (× ATR)", minval = 0, step = 0.05, group = G0,
30     tooltip = "Stops tighter than this get widened so normal noise does not tag them.")
31
32 const string G1 = "1 · Liquidity Sweep"
33 swingLen = input.int(8, "Liquidity swing length", minval = 2, group = G1,
34     tooltip = "Bars on each side that define a swing high/low (a resting liquidity pool). Higher = only major liquidity.")
35 usePdl = input.bool(true, "Add previous-day high / low as liquidity", group = G1)
36 maxPools = input.int(30, "Max pools tracked per side", minval = 5, maxval = 100, group = G1)
37 showPools = input.bool(true, "Show resting liquidity levels", group = G1)
38
39 const string G2 = "2 · V-Shape Recovery"
40 vMaxBars = input.int(6, "Max bars from extreme → recovery", minval = 1, group = G2)
41 vAtrMult = input.float(1.5, "Recovery displacement (× ATR)", minval = 0.1, step = 0.1, group = G2)
42 atrLen = input.int(14, "ATR length", minval = 1, group = G2)
43
44 const string G3 = "3 · CISD + IFVG"
45 alignBars = input.int(12, "Max bars sweep → CISD & IFVG", minval = 1, group = G3)
46 cisdScan = input.int(15, "CISD candle scan depth", minval = 1, maxval = 100, group = G3)
47 fvgLook = input.int(20, "FVG may form up to N bars before sweep", minval = 1, group = G3)
48 fvgMinAtr = input.float(0.1, "Min FVG size (× ATR)", minval = 0, step = 0.05, group = G3)
49
50 const string G4 = "4 · Retest + Second Sweep"
51 retestMax = input.int(16, "Max bars to wait for retest", minval = 1, group = G4)
52 minorLen = input.int(3, "Retest sweep lookback (bars)", minval = 1, group = G4,
53     tooltip = "On the retest, price must take the low (bull) / high (bear) of the previous N bars and close back.")
54
55 const string G5 = "5 · Targets & Management"
56 tpMode = input.string("Fixed R:R", "Target mode", options = ["Fixed R:R", "Liquidity (capped at R:R)"], group = G5,
57     tooltip = "Fixed R:R: TP1 / TP2 exactly at the R multiples below.\nLiquidity (capped): nearest draw on liquidity, never further than the R multiples.")
58 rr1 = input.float(1.0, "TP1 (R)", minval = 0.25, step = 0.25, group = G5, inline = "rr")
59 rr2 = input.float(2.0, "TP2 (R)", minval = 0.5, step = 0.25, group = G5, inline = "rr")
60 requireDol = input.bool(true, "Require draw on liquidity beyond TP1", group = G5,
61     tooltip = "Only enter when an untouched liquidity pool sits at least TP1 away in the trade direction.")
```

```

61 slBuf      = input.float(0.1, "SL buffer (× ATR)", minval = 0, step = 0.05, group = G5)
62 partialPct = input.float(50, "Close % at TP1", minval = 0, maxval = 100, step = 5, group = G5)
63 beAfterTp1 = input.bool(true, "Move SL to entry after TP1", group = G5)
64 tradeMax   = input.int(96, "Close trade after N bars", minval = 1, group = G5)
65
66 const string G6 = "Display"
67 showBull    = input.bool(true, "Bullish setups", group = G6, inline = "d")
68 showBear    = input.bool(true, "Bearish setups", group = G6, inline = "d")
69 bullCol     = input.color(#089981, "Bull", group = G6, inline = "c")
70 bearCol     = input.color(#F23645, "Bear", group = G6, inline = "c")
71 tgtCol      = input.color(#2962FF, "Targets", group = G6, inline = "c")
72 slCol       = input.color(#FF9800, "Stop", group = G6, inline = "c")
73 showStats   = input.bool(true, "Show stats table", group = G6)
74
75 // ----- Core series -----
76 atr         = ta.atr(atrLen)
77 minorLow    = ta.lowest(low, minorLen)[1]
78 minorHigh   = ta.highest(high, minorLen)[1]
79 ph         = ta.pivohigh(high, swingLen, swingLen)
80 pl         = ta.pivotlow(low, swingLen, swingLen)
81
82 // Higher-timeframe bias from the last CLOSED HTF bar (lookahead + [1] = no repaint)
83 htfData() =>
84     float e = ta.ema(close, htfEmaLen)
85     [close[1], e[1], e[2]]
86 [htfClose, htfEma, htfEmaPrev] = request.security(syminfo.tickerid, htfTf, htfData(), lookahead =
    barmerge.lookahead_on)
87 bool bullBias = not useHtf or (htfClose > htfEma and htfEma > htfEmaPrev)
88 bool bearBias = not useHtf or (htfClose < htfEma and htfEma < htfEmaPrev)
89
90 // Killzones
91 bool inKz = not useKz or not na(time(timeframe.period, kz1, kzTz)) or not na(time(timeframe.period, kz2,
    kzTz))
92
93 // Displacement candle (body share of range) for the IFVG inversion
94 float rng    = high - low
95 bool  bullDisp = close > open and rng > 0 and (close - open) / rng >= dispBody
96 bool  bearDisp = close < open and rng > 0 and (open - close) / rng >= dispBody
97
98 // ----- Liquidity pools -----
99 var array<float> bsLPx = array.new<float>() // buy-side liquidity (swing highs)
100 var array<int>  bsLBar = array.new<int>()
101 var array<line> bsLLn  = array.new<line>()
102 var array<float> ssLPx = array.new<float>() // sell-side liquidity (swing lows)
103 var array<int>  sslBar = array.new<int>()
104 var array<line> sslLn  = array.new<line>()
105
106 addPool(array<float> px, array<int> bx, array<line> ln, float p, int b) =>
107     line l = na
108     if showPools
109         l := line.new(b, p, bar_index, p, color = color.new(color.gray, 40), style = line.style_dotted,
            extend = extend.right)
110         px.push(p)
111         bx.push(b)
112         ln.push(l)
113         if px.size() > maxPools
114             px.shift()
115             bx.shift()
116             line.delete(ln.shift())
117
118 // Removes every pool traded through this bar. Returns the most extreme pool that was
119 // swept AND reclaimed (wick through, close back) – d = 1 checks lows, d = -1 checks highs.
120 scanSweeps(array<float> px, array<int> bx, array<line> ln, int d) =>
121     float lvl = na
122     int  lb   = na
123     int  n    = px.size()
124     if n > 0
125         for i = n - 1 to 0
126             float p = px.get(i)
127             bool taken = d == 1 ? low < p : high > p
128             if taken
129                 bool reclaimed = d == 1 ? close > p : close < p
130                 if reclaimed and (na(lvl) or d * (lvl - p) > 0)
131                     lvl := p

```

```

132         lb := bx.get(i)
133         line l = ln.get(i)
134         if not na(l)
135             line.set_x2(l, bar_index)
136             line.set_extend(l, extend.none)
137         px.remove(i)
138         bx.remove(i)
139         ln.remove(i)
140     [lvl, lb]
141
142 // ----- Fair value gaps -----
143 var array<float> bearFvgTop = array.new<float>() // bearish FVGs → become bullish IFVGs
144 var array<float> bearFvgBot = array.new<float>()
145 var array<int> bearFvgBar = array.new<int>()
146 var array<float> bullFvgTop = array.new<float>() // bullish FVGs → become bearish IFVGs
147 var array<float> bullFvgBot = array.new<float>()
148 var array<int> bullFvgBar = array.new<int>()
149
150 addFvg(array<float> top, array<float> bot, array<int> bx, float t, float b, int x) =>
151     top.push(t)
152     bot.push(b)
153     bx.push(x)
154     if top.size() > 100
155         top.shift()
156         bot.shift()
157         bx.shift()
158
159 // Removes every FVG closed through this bar. Returns the most recent inverted FVG that
160 // formed at/after minBar. d = 1 → bearish FVG closed above, d = -1 → bullish FVG closed below.
161 scanInversions(array<float> top, array<float> bot, array<int> bx, int d, int minBar) =>
162     float zt = na
163     float zb = na
164     int zbar = na
165     int n = top.size()
166     if n > 0
167         for i = n - 1 to 0
168             bool inverted = d == 1 ? close > top.get(i) : close < bot.get(i)
169             if inverted
170                 int fb = bx.get(i)
171                 if fb >= minBar and (na(zbar) or fb > zbar)
172                     zt := top.get(i)
173                     zb := bot.get(i)
174                     zbar := fb
175                 top.remove(i)
176                 bot.remove(i)
177                 bx.remove(i)
178     [zt, zb, zbar]
179
180 // CISD reference: open of the earliest candle in the consecutive opposing series that
181 // delivered price into the current extreme (bull: down-close candles, bear: up-close).
182 cisdLevel(int d) =>
183     float lvl = na
184     int lb = na
185     bool run = false
186     for i = 0 to cisdScan
187         bool opp = d == 1 ? close[i] < open[i] : close[i] > open[i]
188         if opp
189             lvl := open[i]
190             lb := bar_index - i
191             run := true
192         else if run
193             break
194     [lvl, lb]
195
196 // Draw on liquidity: nearest two untouched pools in trade direction, at least minDist away.
197 findTargets(int d, float entry, float minDist) =>
198     array<float> pools = d == 1 ? bslPx : sslPx
199     float t1 = na
200     float t2 = na
201     for p in pools
202         if d * (p - entry) >= minDist
203             if na(t1) or d * (t1 - p) > 0
204                 t2 := t1
205                 t1 := p

```

```

206         else if p != t1 and (na(t2) or d * (t2 - p) > 0)
207             t2 := p
208     [t1, t2]
209
210 // ----- Setup state machine -----
211 // st: 0 idle · 1 swept (waiting V + CISD + IFVG) · 2 IFVG armed (waiting retest) · 3 in trade
212 type Setup
213     int    dir
214     int    st      = 0
215     float  lvl
216     int    lvlBar
217     int    sweepBar
218     float  ext
219     int    extBar
220     bool   vDone    = false
221     int    vBar
222     float  vPx
223     float  cisd
224     int    cisdBar
225     bool   cisdDone = false
226     int    cisdConfBar
227     bool   zDone    = false
228     float  zTop
229     float  zBot
230     int    zBar
231     int    armBar
232     float  entry
233     float  sl
234     float  tp1
235     float  tp2
236     float  risk
237     float  r1
238     float  r2
239     bool   tp1Hit   = false
240     int    entryBar
241     box    zBox
242     line   lE
243     line   lS
244     line   lT1
245     line   lT2
246     label  bS
247     label  bT1
248     label  bT2
249
250 method reset(Setup s) =>
251     s.st      := 0
252     s.vDone   := false
253     s.cisdDone := false
254     s.zDone   := false
255     s.tp1Hit  := false
256     s.zBox    := na
257     s.lE      := na
258     s.lS      := na
259     s.lT1     := na
260     s.lT2     := na
261     s.bS      := na
262     s.bT1     := na
263     s.bT2     := na
264     true
265
266 method start(Setup s, float lvl, int lvlBar) =>
267     float adv = s.dir == 1 ? low : high
268     if s.st == 0
269         [c, cb] = cisdLevel(s.dir)
270         s.st     := 1
271         s.lvl     := lvl
272         s.lvlBar  := lvlBar
273         s.sweepBar := bar_index
274         s.ext     := adv
275         s.extBar  := bar_index
276         s.cisd    := c
277         s.cisdBar := cb
278     else if s.st == 1 and s.dir * (s.lvl - lvl) > 0
279         s.lvl := lvl

```

```

280     s.lvlBar := lvlBar
281     true
282
283 method setTradeX(Setup s, int x) =>
284     line.set_x2(s.lE, x)
285     line.set_x2(s.lS, x)
286     line.set_x2(s.lT1, x)
287     line.set_x2(s.lT2, x)
288     label.set_x(s.bS, x)
289     label.set_x(s.bT1, x)
290     label.set_x(s.bT2, x)
291     true
292
293 // bias: HTF bias agrees with this direction · kz: bar is inside a killzone
294 method update(Setup s, float atrV, float minorLvl, float zt, float zb, int zbar, bool bias, bool kz) =>
295     int d = s.dir
296     color c = d == 1 ? bullCol : bearCol
297     float adv = d == 1 ? low : high
298     float fav = d == 1 ? high : low
299     bool evArm = false
300     bool evEntry = false
301     bool evTp1 = false
302     bool evTp2 = false
303     bool evSl = false
304     bool evBe = false
305     float resR = na // realised R when the trade closes on this bar
306     float part = partialPct / 100
307
308     if s.st == 1
309         // New extreme → any partial confirmation is void, rebuild from the new low/high
310         if d * (s.ext - adv) > 0
311             [cl, cb] = cisdLevel(d)
312             s.ext := adv
313             s.extBar := bar_index
314             s.vDone := false
315             s.cisdDone := false
316             s.zDone := false
317             s.cisd := cl
318             s.cisdBar := cb
319             if not na(zbar)
320                 s.zTop := zt
321                 s.zBot := zb
322                 s.zBar := zbar
323                 s.zDone := true
324             if not s.vDone and bar_index - s.extBar <= vMaxBars and d * (fav - s.ext) >= vAtrMult * atrV and
d * (close - s.lvl) > 0
325                 s.vDone := true
326                 s.vBar := bar_index
327                 s.vPx := fav
328             if not s.cisdDone and not na(s.cisd) and d * (close - s.cisd) > 0
329                 s.cisdDone := true
330                 s.cisdConfBar := bar_index
331
332             if s.vDone and s.cisdDone and s.zDone
333                 s.st := 2
334                 s.armBar := bar_index
335                 evArm := true
336                 line.new(s.lvlBar, s.lvl, s.sweepBar, s.lvl, color = c, style = line.style_dashed)
337                 label.new(s.extBar, s.ext, d == 1 ? "SSL sweep" : "BSL sweep", color = color(na), textcolor =
c,
338                     style = d == 1 ? label.style_label_up : label.style_label_down, size = size.small)
339                 label.new(s.vBar, s.vPx, "V", color = color(na), textcolor = c,
340                     style = d == 1 ? label.style_label_down : label.style_label_up, size = size.small)
341                 line.new(s.cisdBar, s.cisd, s.cisdConfBar, s.cisd, color = c, width = 2)
342                 label.new(s.cisdConfBar, s.cisd, "CISD", color = color(na), textcolor = c,
343                     style = label.style_label_left, size = size.small)
344                 s.zBox := box.new(s.zBar, s.zTop, bar_index, s.zBot, border_color = c, bgcolor = color.new(c,
80),
345                     text = "⚡ IFVG", text_color = c, text_size = size.small, text_halign = text.align_right)
346                 else if bar_index - s.sweepBar > alignBars or (not s.vDone and bar_index - s.extBar > vMaxBars)
347                     s.reset()
348
349     else if s.st == 2
350         float zNear = d == 1 ? s.zTop : s.zBot

```

```

351 float zFar = d == 1 ? s.zBot : s.zTop
352 box.set_right(s.zBox, bar_index)
353 if bar_index > s.armBar
354     bool broke = d * (close - zFar) < 0 or d * (adv - s.ext) < 0
355     bool expired = bar_index - s.armBar > retestMax
356     bool touched = d * (adv - zNear) <= 0
357     bool sweep2 = d * (adv - minorLvl) < 0 and d * (close - minorLvl) > 0
358     bool holds = d * (close - zFar) >= 0
359     if broke or expired
360         box.set_bgcolor(s.zBox, color.new(color.gray, 85))
361         box.set_border_color(s.zBox, color.gray)
362         box.set_text_color(s.zBox, color.gray)
363         s.reset()
364     else if touched and sweep2 and holds and bias and kz
365         float entry = close
366         float risk = math.max(math.abs(entry - (adv - d * slBuf * atrV)), minSlAtr * atrV)
367         [dol, dol2] = findTargets(d, entry, rr1 * risk)
368         bool okSize = risk <= maxSlAtr * atrV
369         bool okDol = not requireDol or not na(dol)
370         // A retest that fails the stop-size / DOL filters is skipped; the zone stays armed.
371         if okSize and okDol
372             s.st := 3
373             s.entryBar := bar_index
374             s.entry := entry
375             s.risk := risk
376             s.sl := entry - d * risk
377             float cap1 = entry + d * rr1 * risk
378             float cap2 = entry + d * rr2 * risk
379             if tpMode == "Fixed R:R"
380                 s.tp1 := cap1
381                 s.tp2 := cap2
382             else
383                 [t1, t2] = findTargets(d, entry, 0.5 * risk)
384                 s.tp1 := na(t1) or d * (t1 - cap1) > 0 ? cap1 : t1
385                 s.tp2 := na(t2) or d * (t2 - cap2) > 0 ? cap2 : t2
386                 if d * (s.tp2 - s.tp1) <= 0
387                     s.tp2 := entry + d * math.max(rr2 * risk, math.abs(s.tp1 - entry) + 0.5 *
risk)
388                 s.r1 := math.abs(s.tp1 - entry) / risk
389                 s.r2 := math.abs(s.tp2 - entry) / risk
390                 evEntry := true
391
392                 int x2 = bar_index + 5
393                 string tip = "Entry " + str.toString(s.entry, format.mintick) +
394                     "\nSL " + str.toString(s.sl, format.mintick) +
395                     "\nTP1 " + str.toString(s.tp1, format.mintick) + " (" + str.toString(s.r1,
396                         "#.##") + "R)" +
397                     "\nTP2 " + str.toString(s.tp2, format.mintick) + " (" + str.toString(s.r2,
398                         "#.##") + "R)" +
399                     "\nDraw on liquidity " + (na(dol) ? "none beyond TP1" : str.toString(dol,
format.mintick))
400                 label.new(bar_index, adv, d == 1 ? "BUY" : "SELL", color = c,
401                     textcolor = color.white, style = d == 1 ? label.style_label_up :
label.style_label_down,
402                     size = size.small, tooltip = tip)
403                 box.set_border_width(s.zBox, 2)
404                 s.lE := line.new(bar_index, s.entry, x2, s.entry, color = c, width = 2)
405                 s.lS := line.new(bar_index, s.sl, x2, s.sl, color = slCol, style =
line.style_dashed)
406                 s.lT1 := line.new(bar_index, s.tp1, x2, s.tp1, color = tgtCol, style =
line.style_dashed)
407                 s.lT2 := line.new(bar_index, s.tp2, x2, s.tp2, color = tgtCol)
408                 s.bS := label.new(x2, s.sl, "SL", color = color(na), textcolor = slCol,
409                     style = label.style_label_left, size = size.small)
410                 s.bT1 := label.new(x2, s.tp1, "TP1 " + str.toString(s.r1, "#.##") + "R", color =
color(na),
411                     textcolor = tgtCol, style = label.style_label_left, size = size.small)
412                 s.bT2 := label.new(x2, s.tp2, "TP2 " + str.toString(s.r2, "#.##") + "R", color =
color(na),
413                     textcolor = tgtCol, style = label.style_label_left, size = size.small)
414
415 else if s.st == 3
416     if bar_index > s.entryBar
417         s.setTradeX(bar_index + 5)

```

```

416     bool hitSl = d * (adv - s.sl) <= 0
417     bool hitTp1 = d * (fav - s.tp1) >= 0
418     bool hitTp2 = d * (fav - s.tp2) >= 0
419     // Conservative: if SL and a target print on the same bar, the stop is assumed first
420     if hitSl
421         if s.tp1Hit
422             evBe := true
423             resR := part * s.r1 + (1 - part) * d * (s.sl - s.entry) / s.risk
424         else
425             evSl := true
426             resR := -1.0
427         label.new(bar_index, adv, s.tp1Hit ? (beAfterTp1 ? "BE" : "SL after TP1") : "SL x", color
= color(na),
428             textcolor = slCol, style = d == 1 ? label.style_label_up : label.style_label_down,
size = size.small)
429         s.setTradeX(bar_index)
430         s.reset()
431     else if hitTp2
432         if not s.tp1Hit
433             evTp1 := true
434             evTp2 := true
435             resR := part * s.r1 + (1 - part) * s.r2
436             label.new(bar_index, s.tp2, "TP2 ✓", color = color(na), textcolor = tgtCol,
437                 style = d == 1 ? label.style_label_down : label.style_label_up, size = size.small)
438             s.setTradeX(bar_index)
439             s.reset()
440         else
441             if hitTp1 and not s.tp1Hit
442                 s.tp1Hit := true
443                 evTp1 := true
444                 label.new(bar_index, s.tp1, "TP1 ✓", color = color(na), textcolor = tgtCol,
445                     style = d == 1 ? label.style_label_down : label.style_label_up, size =
size.small)
446             if beAfterTp1
447                 s.sl := s.entry
448                 line.set_y1(s.lS, s.entry)
449                 line.set_y2(s.lS, s.entry)
450                 label.set_y(s.bS, s.entry)
451                 label.set_text(s.bS, "SL → BE")
452             if bar_index - s.entryBar >= tradeMax
453                 float openR = d * (close - s.entry) / s.risk
454                 resR := s.tp1Hit ? part * s.r1 + (1 - part) * openR : openR
455                 label.new(bar_index, close, "Time exit " + str.tostring(resR, "#.##") + "R", color =
color(na),
456                     textcolor = color.gray, style = label.style_label_left, size = size.small)
457                 s.setTradeX(bar_index)
458                 s.reset()
459
460     [evArm, evEntry, evTp1, evTp2, evSl, evBe, resR]
461
462 // ----- Engine -----
463 var Setup bull = Setup.new(1)
464 var Setup bear = Setup.new(-1)
465
466 // 1 · register new liquidity pools
467 if not na(ph)
468     addPool(bslPx, bslBar, bslLn, ph, bar_index - swingLen)
469 if not na(pl)
470     addPool(sslPx, sslBar, sslLn, pl, bar_index - swingLen)
471
472 // previous-day high / low become liquidity pools at the start of each new day
473 var float dH = na
474 var float dL = na
475 var int dHB = na
476 var int dLB = na
477 if timeframe.change("D")
478     if usePdl and not na(dH)
479         addPool(bslPx, bslBar, bslLn, dH, dHB)
480         addPool(sslPx, sslBar, sslLn, dL, dLB)
481     dH := high
482     dHB := bar_index
483     dL := low
484     dLB := bar_index
485 else

```

```

486     if na(dH) or high > dH
487         dH := high
488         dHB := bar_index
489     if na(dL) or low < dL
490         dL := low
491         dLB := bar_index
492
493 // 2 · liquidity sweeps start (or deepen) a setup
494 [sslLvl, sslB] = scanSweeps(sslPx, sslBar, sslLn, 1)
495 [bslLvl, bslB] = scanSweeps(bslPx, bslBar, bslLn, -1)
496 if showBull and bullBias and not na(sslLvl)
497     bull.start(sslLvl, sslB)
498 if showBear and bearBias and not na(bslLvl)
499     bear.start(bslLvl, bslB)
500
501 // 3 · FVG inversions (only FVGs formed around the sweep, inverted by a displacement candle)
502 int bullMin = bull.st == 1 and bullDisp ? bull.sweepBar - fvgLook : bar_index + 1
503 int bearMin = bear.st == 1 and bearDisp ? bear.sweepBar - fvgLook : bar_index + 1
504 [bzt, bzb, bzbar] = scanInversions(bearFvgTop, bearFvgBot, bearFvgBar, 1, bullMin)
505 [szt, szb, szbar] = scanInversions(bullFvgTop, bullFvgBot, bullFvgBar, -1, bearMin)
506
507 if high < low[2] and low[2] - high >= fvgMinAtr * atr
508     addFvg(bearFvgTop, bearFvgBot, bearFvgBar, low[2], high, bar_index - 1)
509 if low > high[2] and low - high[2] >= fvgMinAtr * atr
510     addFvg(bullFvgTop, bullFvgBot, bullFvgBar, low, high[2], bar_index - 1)
511
512 // 4 · advance state machines
513 [bArm, bEntry, bTp1, bTp2, bSl, bBe, bR] = bull.update(atr, minorLow, bzt, bzb, bzbar, bullBias, inKz)
514 [sArm, sEntry, sTp1, sTp2, sSl, sBe, sR] = bear.update(atr, minorHigh, szt, szb, szbar, bearBias, inKz)
515
516 // ----- Stats -----
517 var int    nTrades = 0
518 var int    nClosed = 0
519 var int    nWin    = 0
520 var int    nTp1    = 0
521 var int    nTp2    = 0
522 var int    nLoss    = 0
523 var float  netR     = 0.0
524 nTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
525 nTp1    += (bTp1 ? 1 : 0) + (sTp1 ? 1 : 0)
526 nTp2    += (bTp2 ? 1 : 0) + (sTp2 ? 1 : 0)
527 nLoss    += (bSl ? 1 : 0) + (sSl ? 1 : 0)
528 for r in array.from(bR, sR)
529     if not na(r)
530         nClosed += 1
531         nWin    += r > 0 ? 1 : 0
532         netR    += r
533
534 statRow(table t, int row, string k, string v, color c) =>
535     table.cell(t, 0, row, k, text_color = color.silver, text_size = size.small, text_halign =
text.align_left)
536     table.cell(t, 1, row, v, text_color = c, text_size = size.small, text_halign = text.align_right)
537
538 pct(int a, int b) => b > 0 ? str.toString(100.0 * a / b, "#") + "%" : "-"
539
540 var table stats = table.new(position.top_right, 2, 8, bgcolor = color.new(#131722, 10), border_width = 1)
541 if showStats and barstate.islast
542     table.cell(stats, 0, 0, "IFVG Sniper · " + timeframe.period + (timeframe.period == "15" ? "" : "
(tuned for 15)"),
543         text_color = color.white, text_size = size.small)
544     table.merge_cells(stats, 0, 0, 1, 0)
545     statRow(stats, 1, "Trades closed", str.toString(nClosed) + " / " + str.toString(nTrades),
color.white)
546     statRow(stats, 2, "Win rate (R > 0)", pct(nWin, nClosed), tgtCol)
547     statRow(stats, 3, "TP1 reached", pct(nTp1, nClosed), tgtCol)
548     statRow(stats, 4, "TP2 reached", pct(nTp2, nClosed), tgtCol)
549     statRow(stats, 5, "Stopped before TP1", pct(nLoss, nClosed), slCol)
550     statRow(stats, 6, "Net R", str.toString(netR, "###"), netR >= 0 ? tgtCol : slCol)
551     statRow(stats, 7, "Avg R / trade", nClosed > 0 ? str.toString(netR / nClosed, "###") : "-", netR >=
0 ? tgtCol : slCol)
552
553 // ----- Data window -----
554 plot(bull.st == 3 ? bull.entry : na, "Bull entry", bullCol, display = display.data_window)
555 plot(bull.st == 3 ? bull.sl : na, "Bull SL", slCol, display = display.data_window)

```

```

556 plot(bull.st == 3 ? bull.tp1 : na, "Bull TP1", tgtCol, display = display.data_window)
557 plot(bull.st == 3 ? bull.tp2 : na, "Bull TP2", tgtCol, display = display.data_window)
558 plot(bear.st == 3 ? bear.entry : na, "Bear entry", bearCol, display = display.data_window)
559 plot(bear.st == 3 ? bear.sl : na, "Bear SL", slCol, display = display.data_window)
560 plot(bear.st == 3 ? bear.tp1 : na, "Bear TP1", tgtCol, display = display.data_window)
561 plot(bear.st == 3 ? bear.tp2 : na, "Bear TP2", tgtCol, display = display.data_window)
562
563 // ----- Alerts -----
564 alertcondition(bArm, "Bull IFVG armed", "IFVG Sniper: bullish sweep + V + CISD + IFVG armed on
{{ticker}} ({{interval}})")
565 alertcondition(bEntry, "Bull entry", "IFVG Sniper: BUY on {{ticker}} @ {{close}} - IFVG retest +
sweep")
566 alertcondition(bTp1, "Bull TP1", "IFVG Sniper: bullish TP1 hit on {{ticker}}")
567 alertcondition(bTp2, "Bull TP2", "IFVG Sniper: bullish TP2 hit on {{ticker}}")
568 alertcondition(bSl, "Bull stop", "IFVG Sniper: bullish stop hit on {{ticker}}")
569 alertcondition(sArm, "Bear IFVG armed", "IFVG Sniper: bearish sweep + V + CISD + IFVG armed on
{{ticker}} ({{interval}})")
570 alertcondition(sEntry, "Bear entry", "IFVG Sniper: SELL on {{ticker}} @ {{close}} - IFVG retest +
sweep")
571 alertcondition(sTp1, "Bear TP1", "IFVG Sniper: bearish TP1 hit on {{ticker}}")
572 alertcondition(sTp2, "Bear TP2", "IFVG Sniper: bearish TP2 hit on {{ticker}}")
573 alertcondition(sSl, "Bear stop", "IFVG Sniper: bearish stop hit on {{ticker}}")
574
575 if bEntry
576     alert("IFVG Sniper BUY " + syminfo.ticker + " | entry " + str.tostring(bull.entry, format.mintick) +
577         " | SL " + str.tostring(bull.sl, format.mintick) + " | TP1 " + str.tostring(bull.tp1,
format.mintick) +
578         " | TP2 " + str.tostring(bull.tp2, format.mintick), alert.freq_once_per_bar_close)
579 if sEntry
580     alert("IFVG Sniper SELL " + syminfo.ticker + " | entry " + str.tostring(bear.entry, format.mintick) +
581         " | SL " + str.tostring(bear.sl, format.mintick) + " | TP1 " + str.tostring(bear.tp1,
format.mintick) +
582         " | TP2 " + str.tostring(bear.tp2, format.mintick), alert.freq_once_per_bar_close)
583

```